The Termination Problem for Session Types on Objects: Analysis, Options, and Trade-offs

Alexandre Zua Caldeira
LASIGE, Faculdade de Ciências, Universidade de Lisboa

March 3, 2026

Abstract

Session types describe communication protocols on objects as sequences of method calls, branches, selections, and recursive loops. A fundamental question is whether a recursive session type *terminates*: does every protocol execution eventually reach the terminal state **end**? This document analyses the termination problem in the context of our lattice-based framework for session types on objects. We formalise the current syntactic termination check, characterise the class of types it rejects, examine the consequences of non-termination for lattice structure and tool pipelines, and survey four alternative policies ranging from strict enforcement to full permissiveness. We conclude with a recommendation: termination should be enforced by default but overridable via an explicit `skipTermination` annotation, allowing researchers and tool developers to explore non-terminating protocols without compromising the safety guarantees of the default pipeline.

Contents

1 Introduction **2**

2 Formal Framework **2**
2.1 Session Type Syntax . . . 2
2.2 Termination . . . 3
2.3 Properties of the Termination Check . . . 3

3 Impact on Lattice Structure **4**
3.1 The Bottom Element Problem . . . 4
3.2 The SCC Quotient . . . 4

4 Current Tool Behaviour **5**
4.1 Pipeline Architecture . . . 5
4.2 Asymmetry Between Processor and CLI . . . 5
4.3 The `skipTermination` Mechanism . . . 5
4.4 Guarding Against the Degenerate Case . . . 6

5 Alternative Termination Policies **6**
5.1 Option A: Strict Enforcement (Status Quo Default) . . . 6
5.2 Option B: Opt-in Skip (Current Implementation) . . . 6
5.3 Option C: Informational Only (No Gate) . . . 7
5.4 Option D: Stronger Termination (All Paths Must Exit) . . . 7

6 Termination and Parallel Composition **8**

7 Related Considerations **9**
7.1 Coinductive Session Types . . . 9
7.2 Fairness . . . 9
7.3 Productivity . . . 9

8 Comparison Summary **10**

9 Recommendation **10**

10 Future Work **10**

1 Introduction

Recursion is indispensable for expressing protocols that repeat: an iterator loops over elements, a server handles requests in a cycle, a transaction retries on failure. The standard mechanism in session type theory is the μ-binder:

$$\mu X.\, \&\{\mathit{next} : X,\ \mathit{done} : \mathbf{end}\}$$

describes a protocol that repeatedly offers `next` (looping) or `done` (terminating). The recursion variable X marks the point to which control returns.

A session type is *terminating* if, for every recursive sub-expression $\mu X.\, S$, there exists at least one syntactic path from the root of S to **end** that does not pass through X. This guarantees that the protocol *can* complete, though it does not force every execution to do so — a branch may loop indefinitely as long as some alternative exits.

Non-terminating types such as $\mu X.\, X$ or $\mu X.\, \&\{a : X\}$ have no path to **end**. These types create *degenerate* state spaces: the bottom element (the terminal state) is unreachable, lattice structure breaks down, and downstream analyses (lattice checking, thread safety, typestate verification) produce meaningless or erroneous results.

The purpose of this document is to:

(i) Formalise the termination property and the current checking algorithm (Section 2).

(ii) Characterise the impact of non-termination on lattice structure (Section 3).

(iii) Describe the current tool behaviour and the recently introduced `skipTermination` mechanism (Section 4).

(iv) Survey four alternative termination policies with their trade-offs (Section 5).

(v) Discuss the parallel constructor, where termination is most critical (Section 6).

(vi) Offer a recommendation (Section 9).

2 Formal Framework

2.1 Session Type Syntax

We work with the following grammar for session types on objects:

Definition 2.1 (Session type syntax)**.**

$$S \ ::= \ \&\{m_1 : S_1, \ldots, m_n : S_n\} \mid \oplus\{l_1 : S_1, \ldots, l_n : S_n\} \mid S_1 \parallel S_2 \mid \mu X.\, S \mid X \mid \mathbf{end} \mid S_1 \,.\, S_2$$

where $\&\{\cdot\}$ is branch (external choice), $\oplus\{\cdot\}$ is selection (internal choice), $\parallel$ is parallel composition, $\mu X.\, S$ is recursion binding variable X, and **end** is the terminated protocol. Sequencing $S_1 \,.\, S_2$ is syntactic sugar for a single-method branch.

2.2 Termination

Definition 2.2 (Exit path)**.** Given a session type node S and a forbidden variable X, we say S has an *exit path avoiding* X, written $\mathit{exit}(S, X)$, if and only if:

$$\frac{}{\mathit{exit}(\mathbf{end}, X)} \qquad \frac{Y \neq X}{\mathit{exit}(Y, X)} \qquad \frac{\exists\, i.\ \mathit{exit}(S_i, X)}{\mathit{exit}(\&\{m_1 : S_1, \ldots, m_n : S_n\}, X)}$$

$$\frac{\exists\, i.\ \mathit{exit}(S_i, X)}{\mathit{exit}(\oplus\{l_1 : S_1, \ldots, l_n : S_n\}, X)} \qquad \frac{\mathit{exit}(S_1, X) \qquad \mathit{exit}(S_2, X)}{\mathit{exit}(S_1 \,.\, S_2, X)} \qquad \frac{\mathit{exit}(S_1, X) \qquad \mathit{exit}(S_2, X)}{\mathit{exit}(S_1 \parallel S_2, X)}$$

$$\frac{\mathit{exit}(S, X)}{\mathit{exit}(\mu Y.\, S, X)}$$

The key asymmetry is between branching/selection (existential: *some* branch must exit) and sequencing/parallel (universal: *both* components must exit). The justification is that branching represents a choice — the protocol is well-formed if at least one alternative leads to completion. Sequencing and parallel represent obligations — both parts must independently be capable of completing.

Definition 2.3 (Terminating session type)**.** A session type S is *terminating* if, for every subexpression $\mu X.\, S'$ occurring in S, we have $\mathit{exit}(S', X)$.

Example 2.4 (Terminating types)*.* The following are terminating:

$\mu X.\, \&\{\mathit{read} : X,\ \mathit{done} : \mathbf{end}\}$	(exit via `done`)
$\mu X.\, \&\{\mathit{hasNext} : \oplus\{\mathit{TRUE} : \&\{\mathit{next} : X\},\ \mathit{FALSE} : \mathbf{end}\}\}$	(Java Iterator)
$\mu X.\, \&\{a : \mu Y.\, \&\{b : Y,\ c : X\},\ d : \mathbf{end}\}$	(nested recursion)

In the nested case, the inner $\mu Y.\, \cdot$ has exit via branch c (which goes to X, not Y), and the outer $\mu X.\, \cdot$ has exit via branch d.

Example 2.5 (Non-terminating types)*.* The following are *not* terminating:

$\mu X.\, X$	(immediate divergence)
$\mu X.\, \&\{a : X\}$	(single looping branch)
$\mu X.\, \&\{a : X,\ b : X\}$	(all branches loop)
$\mu X.\, \&\{a : \oplus\{\mathit{OK} : X,\ \mathit{ERR} : X\}\}$	(nested, all paths loop)

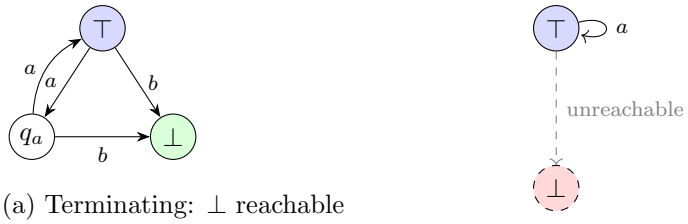
2.3 Properties of the Termination Check

Proposition 2.6 (Decidability)**.** *The termination check is decidable for all session types with finite branching.*

Proof. The function $\mathit{exit}(S, X)$ performs a single structural recursion over the finite AST of S. No fixed-point iteration, unfolding, or state space construction is required. The check runs in $O(|S|)$ time, where $|S|$ is the size of the syntax tree. □

Proposition 2.7 (Soundness)**.** *If S is terminating, then in the state space $\mathcal{L}(S)$, the bottom state $\bot$ (corresponding to* **end***) is reachable from the top state $\top$ (the initial state).*

Remark 2.8 (Existential, not universal)*.* The termination check guarantees that *some* execution path reaches **end**, not that *all* paths do. In a type like $\mu X.\, \&\{a : X,\ b : \mathbf{end}\}$, the branch a loops forever if always chosen. The check certifies that the protocol *can* terminate (via b), not that it *must*. This is the standard approach in session type literature: termination is a *liveness* property of the protocol specification, not a guarantee about runtime behaviour.



(a) Terminating: ⊥ reachable

(b) Non-terminating: ⊥ unreachable

Figure 1: State spaces for $\mu X.\&\{a : X, b : \mathbf{end}\}$ (terminating) and $\mu X.\&\{a : X\}$ (non-terminating). In (b), the bottom state exists in the representation but is unreachable; the state space restricted to reachable states is $\{\top\}$ with a self-loop, which is not a lattice (no distinct bottom).

3 Impact on Lattice Structure

Our central thesis is that session type state spaces form lattices. Termination is a precondition for this property.

3.1 The Bottom Element Problem

Definition 3.1 (State space)**.** The state space $\mathcal{L}(S)$ of a session type S is a directed graph $(Q, \delta, \top, \bot)$ where Q is a finite set of states, $\delta \subseteq Q \times \Sigma \times Q$ is the transition relation labelled by method names, $\top \in Q$ is the initial state, and $\bot \in Q$ is the terminal state corresponding to **end**.

The reachability ordering $s_1 \geq s_2 \iff s_1 \to^* s_2$ makes $\mathcal{L}(S)$ a poset with $\top$ as the greatest element. For $\mathcal{L}(S)$ to be a lattice, it must also have a least element $\bot$ reachable from every state.

Proposition 3.2 (Non-termination breaks lattice structure)**.** *If S is non-terminating, then $\bot \notin Q_{reach}$, where $Q_{reach} = \{q \in Q \mid \top \to^* q\}$ is the set of reachable states. Consequently, $\mathcal{L}(S)$ restricted to reachable states has no bottom element and is not a lattice.*

Proof. Consider $S = \mu X.\&\{a_1 : S_1, \ldots, a_n : S_n\}$ where every S_i eventually leads back to X without passing through **end**. The state space builder allocates $\bot$ as the **end**-state but no transition path from $\top$ ever reaches it. Filtering to reachable states excludes $\bot$. Without a bottom element, the poset cannot be a lattice. □

This is not merely a theoretical concern. In our implementation (Reticulate and Bica Reborn), attempting to check lattice properties on a degenerate state space previously caused a `NullPointerException` when the lattice checker looked up the SCC containing $\bot$ — because $\bot$ was not in the state set.

3.2 The SCC Quotient

For terminating recursive types, the state space may contain cycles (from recursion). Our lattice checker handles this by quotienting by strongly connected components (SCCs): states in the same SCC are collapsed to a single node, yielding a DAG on which meets and joins can be computed.

For non-terminating types, the SCC quotient is degenerate: the entire reachable state space may be a single SCC (e.g., $\mu X.\&\{a : X\}$ produces one state with a self-loop, which is its own SCC). The quotient has one node, no bottom distinct from top, and the lattice check trivially fails.

4 Current Tool Behaviour

4.1 Pipeline Architecture

Both Reticulate (Python) and Bica Reborn (Java) implement a multi-stage pipeline for session type analysis:

Stage	Analysis	Depends on
1	Parse	—
2	Termination check	AST
3	WF-Par check	AST
4	Build state space	AST
5	Lattice check	State space
6	Thread safety	AST + source
7	Typestate checking	State space + source

The critical design question is: *does termination failure at Stage 2 block the remainder of the pipeline?*

4.2 Asymmetry Between Processor and CLI

Historically, the annotation processor (Bica Reborn) and the CLI/web tools handled termination failure differently:

Annotation processor (hard gate)
If termination fails, the processor emits a *compiler error* and returns `null`. No state space is built. No lattice check, thread safety, or typestate verification occurs. The rationale: a non-terminating session type is a specification error; there is no point analysing a broken protocol.

CLI and web tools (informational)
The CLI builds the state space *first* (Stage 4), then runs termination as an informational check (Stage 2 conceptually, but executed after Stage 4). The result is displayed with a check mark (✓) or cross (×), but does not block the output. This allows users to inspect the state space and lattice properties of non-terminating types for research purposes.

This asymmetry created a usability problem: researchers who wanted to explore non-terminating protocols could do so in the CLI but not when using Bica Reborn's annotation processor in their Java code.

4.3 The `skipTermination` Mechanism

To resolve this asymmetry, we introduced an opt-in escape hatch at two levels:

Annotation level. The `@Session` annotation now accepts an optional `skipTermination` attribute:

```
@Session(value="rec X . X", skipTermination=true)
```

When `skipTermination=true` and the type is non-terminating:

- The termination check emits a *warning* instead of an error.
- The WF-Par check also emits a warning if it fails.
- The pipeline continues: state space is built, lattice is checked.

- If the lattice check fails *as a consequence of non-termination* (degenerate bottom), this too is downgraded to a warning.
- The class compiles successfully; downstream typestate checking proceeds with the (possibly degenerate) state space.

CLI level. The `--skip-termination` flag achieves the same effect. The summary output shows a warning symbol (△) and the word "skipped" instead of the cross mark.

4.4 Guarding Against the Degenerate Case

The introduction of `skipTermination` exposed a latent bug: the lattice checker assumed that $\bot \in Q$, but for non-terminating types, $\bot \notin Q_{reach}$. Looking up $\bot$ in the SCC map produced a `null` value, causing a `NullPointerException` on unboxing.

The fix was a guard at the entry of the lattice checker:

if $\bot \notin Q$ then return `LatticeResult(false, false, false, false, false, 0, null, {})`

This degenerate result correctly reports: not a lattice, no top (in the lattice sense), no bottom, no meets, no joins. It is safe to construct and will not trigger null pointer errors in downstream code.

5 Alternative Termination Policies

We identify four distinct policies for handling termination in the session type pipeline, ranging from strictest to most permissive.

5.1 Option A: Strict Enforcement (Status Quo Default)

Policy. Termination failure is a hard error. Non-terminating types are rejected at the earliest possible stage; no downstream analysis is performed.

Pros.

- **Strongest guarantees.** Every accepted type has a reachable $\bot$, a well-formed lattice, and meaningful thread safety results.
- **Fail-fast.** Errors are caught before expensive state space construction.
- **Simplicity.** Downstream stages can assume well-formedness without defensive checks.

Cons.

- **Inflexible.** Users cannot explore the partial structure of non-terminating types.
- **Blocks research.** Some session type theories (e.g., fair subtyping, coinductive session types) deliberately work with non-terminating types.

5.2 Option B: Opt-in Skip (Current Implementation)

Policy. Termination is enforced by default. Users may opt out via `skipTermination=true` (annotation) or `--skip-termination` (CLI). When skipped, termination failure produces warnings; the pipeline continues with best-effort results.

Pros.

- **Safe default.** Users who do not set the flag get full enforcement (Option A behaviour).
- **Explicit opt-in.** The flag is a clear signal that the user understands the type may be non-terminating.
- **Gradual exploration.** Useful for prototyping: write a type, skip termination, inspect the state space, then add exit paths.
- **Research-friendly.** Enables studying lattice failures and degenerate state spaces without modifying the tool's core logic.

Cons.

- **Downstream noise.** The lattice check, thread safety, and typestate checker produce results on degenerate state spaces that may be confusing or misleading.
- **Annotation pollution.** The `skipTermination` attribute is visible in production code, possibly masking bugs.
- **Cascading warnings.** A single non-terminating recursion variable can trigger warnings at termination, WF-Par, and lattice stages — the user sees three warnings for one root cause.

5.3 Option C: Informational Only (No Gate)

Policy. Termination is always checked but never blocks the pipeline. Non-terminating types produce warnings (or notes) and the pipeline continues unconditionally.

Pros.

- **Uniform behaviour.** CLI, web, and annotation processor all behave the same way.
- **No annotation noise.** No need for `skipTermination` attributes.
- **Maximum information.** The user always sees the full analysis, even for broken types.

Cons.

- **Weaker safety net.** Non-terminating types silently pass through the compiler. The lattice check will catch the problem, but the error message ("not a lattice") is less informative than "non-terminating recursion variable X".
- **False sense of security.** A user may see "verified" in the output and not notice the termination warning.
- **Breaks the lattice invariant.** Downstream tools and generated tests may assume lattice structure that does not hold.

5.4 Option D: Stronger Termination (All Paths Must Exit)

Policy. Strengthen the termination check to require that *every* path from the root of a recursive body reaches **end**, not just some path. Formally, replace the existential quantifier in the branch/selection rules with a universal one:

$$\frac{\forall\, i.\ \mathit{exit}_{\forall}(S_i, X)}{\mathit{exit}_{\forall}(\&\{m_1 : S_1, \ldots, m_n : S_n\}, X)}$$

Pros.

- **Strongest guarantee.** Every execution path is guaranteed to terminate, not just some.
- **Prevents livelock.** A protocol like $\mu X. \&\{a : X,\ b : \mathbf{end}\}$ would be rejected because branch a loops.

Cons.

- **Too restrictive.** Rejects many useful protocols. The Java Iterator pattern $\mu X. \&\{\mathit{hasNext} : \oplus\{\mathit{TRUE} : \&\{\mathit{next} : X\},\ \mathit{FALSE} : \mathbf{end}\}\}$ would be rejected because the TRUE branch loops.
- **Not standard.** The session type literature uniformly uses the existential (some-path) formulation. Switching to universal would break compatibility with existing benchmarks.
- **Conflates specification with implementation.** The session type specifies the *protocol shape*, not the runtime scheduling. Whether branch a is chosen infinitely often is a fairness property, not a type property.

6 Termination and Parallel Composition

The parallel constructor $S_1 \parallel S_2$ is where termination is most critical. The product lattice construction requires both branches to have well-defined bottom elements:

Definition 6.1 (Product state space)**.**

$$\mathcal{L}(S_1 \parallel S_2) \;=\; \mathcal{L}(S_1) \times \mathcal{L}(S_2)$$

with componentwise ordering: $(q_1, q_2) \geq (q_1', q_2') \iff q_1 \geq q_1'$ and $q_2 \geq q_2'$. The top is $(\top_1, \top_2)$ and the bottom is $(\bot_1, \bot_2)$.

Proposition 6.2 (Product lattice requires termination)**.** *If either S_1 or S_2 is non-terminating, then $\mathcal{L}(S_1 \parallel S_2)$ has no bottom element and is not a lattice.*

Proof. Without loss of generality, assume $\bot_1 \notin Q^{(1)}_{reach}$. Then for any $q_2 \in Q^{(2)}$, the pair $(\bot_1, q_2)$ is unreachable in the product. In particular, $(\bot_1, \bot_2)$ is unreachable, so the product has no bottom. □

The WF-Par well-formedness check enforces three conditions on every $S_1 \parallel S_2$:

Definition 6.3 (WF-Par)**.** A parallel composition $S_1 \parallel S_2$ is *well-formed* if:

(WF1) **Termination:** Both S_1 and S_2 are terminating.

(WF2) **Variable scoping:** $\mathit{fv}(S_1) \cap \mathit{bv}(S_2) = \emptyset$ and $\mathit{fv}(S_2) \cap \mathit{bv}(S_1) = \emptyset$. No cross-branch recursion variables.

(WF3) **No nesting:** Neither S_1 nor S_2 contains a $\parallel$ sub-expression.

Condition (WF1) is a direct application of the termination check. Condition (WF2) prevents shared recursion variables that would entangle the two branches' control flow. Condition (WF3) is a current restriction that avoids n-ary parallelism complexity; it may be relaxed in future work.

Property	Option A (Strict)	Option B (Opt-in skip)	Option C (Info only)	Option D (All paths)
$\mu X.\,\&\{a : X, b : \mathbf{end}\}$	✓	✓	✓	×
$\mu X.\,\&\{a : X\}$	×	skip	warn	×
$\mu X.\,X$	×	skip	warn	×
Java Iterator	✓	✓	✓	×
Lattice guaranteed	yes	default yes	no	yes
Safe default	yes	yes	no	yes
Research-friendly	no	yes	yes	no
Std. compatible	yes	yes	yes	no

Table 1: Comparison of termination policies. "skip" means the type is accepted with warnings when the flag is set; "warn" means always accepted with a warning.

7 Related Considerations

7.1 Coinductive Session Types

Some session type theories adopt a *coinductive* interpretation of recursive types, where $\mu X.\,X$ is a valid (though degenerate) type representing an infinite, never-completing protocol. In this setting, termination is not required; instead, progress and type preservation are ensured by other means (e.g., fair subtyping).

Our framework is *inductive*: we require finite reachability and lattice structure. The coinductive approach is incompatible with our lattice-based analysis but represents a legitimate alternative foundation. The `skipTermination` mechanism allows users to experiment with coinductive-style types within our inductive toolchain, albeit without full lattice guarantees.

7.2 Fairness

Our termination check is strictly syntactic: it checks whether an exit path *exists*, not whether it will be *taken*. This is intentional. Runtime fairness — ensuring that an available exit is eventually taken — is a separate concern that belongs to the execution semantics, not the type system.

A stronger property, *strong termination* (all paths exit), would subsume fairness at the type level but at the cost of rejecting useful protocols (see Option D in Section 5.4).

7.3 Productivity

An alternative to termination is *productivity*: the requirement that a recursive type always makes progress (produces an observable action) before recurring. The type $\mu X.\,X$ is unproductive (immediate self-reference with no action), while $\mu X.\,\&\{a : X\}$ is productive (the method a is an observable action before looping).

Productivity is a weaker property than termination: productive types may loop forever, but they always "do something" before looping. This is relevant for stream-like protocols where infinite operation is the intended behaviour.

Our current check does not distinguish productive from unproductive non-terminating types. Adding a productivity check could enable a middle ground: reject unproductive types ($\mu X.\,X$) while allowing productive infinite protocols ($\mu X.\,\&\{a : X\}$) under the `skipTermination` flag.

8 Comparison Summary

9 Recommendation

We recommend **Option B** (opt-in skip), which is the current implementation. The rationale is:

1. **Safe by default.** The vast majority of users write terminating protocols. These users benefit from strict enforcement (Option A behaviour) without any configuration.

2. **Explicit escape hatch.** Researchers and advanced users who need to explore non-terminating types can opt in via `skipTermination=true`. The annotation serves as documentation: it signals to future readers that the type is intentionally non-terminating.

3. **Warning trail.** When termination is skipped, the pipeline produces a clear trail of warnings (termination skipped, lattice not a lattice, etc.). This ensures the user is aware of the consequences without being blocked.

4. **Forward compatibility.** If we later add productivity checking (Section 7.3) or coinductive mode (Section 7.1), the `skipTermination` flag provides a natural migration path: it could be extended to `terminationPolicy = {STRICT, PRODUCTIVE, SKIP}`.

The key principle is: *the type system should be conservative by default but not dogmatic.* Researchers must be able to probe the boundaries of the theory, and practitioners must be able to prototype incrementally. An opt-in escape hatch achieves both goals.

10 Future Work

1. **Productivity check.** Add a check that distinguishes productive non-termination ($\mu X.\,\&\{a : X\}$) from unproductive divergence ($\mu X.\,X$). Productive types could be allowed in certain contexts (e.g., stream protocols) without requiring `skipTermination`.

2. **Partial lattice analysis.** When the bottom is unreachable, the reachable fragment of the state space may still have useful lattice-like properties (e.g., all pairwise meets exist among reachable states). Reporting these partial results could be informative for research.

3. **Termination policy enum.** Extend `skipTermination` (boolean) to a richer `terminationPolicy` enum: `STRICT` (current default), `PRODUCTIVE` (allow productive non-termination), `COINDUCTIVE` (allow all recursive types, check progress instead), `SKIP` (current skip behaviour).

4. **Fair termination.** Investigate whether runtime fairness assumptions can be statically approximated, enabling a middle ground between existential termination (current) and universal termination (Option D).

5. **Relaxing WF-Par (WF3).** The no-nesting restriction on parallel is orthogonal to termination but interacts with it: nested parallelism multiplies the number of branches that must independently terminate. Relaxing this restriction requires revisiting the termination check for deeply nested $\|$ expressions.

References

[1] K. Honda, V.T. Vasconcelos, and M. Kubo. Language primitives and type discipline for structured communication-based programming. In *ESOP*, LNCS 1381, pp. 122–138. Springer, 1998.

[2] S.J. Gay and V.T. Vasconcelos. Linear type theory for asynchronous session types. *J. Functional Programming*, 20(1):19–50, 2010.

[3] D. Kouzapas, O. Dardha, R. Perera, and S.J. Gay. Typechecking protocols with Mungo and StMungo. In *PLACES*, EPTCS 203, pp. 146–159, 2016.

[4] O. Dardha, E. Giachino, and D. Sangiorgi. Session types revisited. *Inf. Comput.*, 256:253–286, 2017.

[5] L. Ciccone, F. Dagnino, and E. Zucca. Fair subtyping for multi-party session types. In *ECOOP*, LIPIcs 222, 2022.

[6] A. Zua Caldeira and V.T. Vasconcelos. Session type state spaces form lattices. Manuscript, 2025.